Integridade Referencial em Bancos de Dados Relacionais

Tutorial com MySQL, exemplos, exercícios e soluções

Professor Márcio Fantini - coltec-ufmg-br

Outubro de 2026

Sumário

1 Conceitos iniciais: tabela pai e tabela filha **2**
1.1 Tabela pai (tabela referenciada) . . . 2
1.2 Tabela filha (tabela que referencia) . . . 2
1.3 Exemplo introdutório . . . 2

2 Definições **3**
2.1 Três categorias de integridade . . . 3
2.1.1 Integridade de domínio . . . 3
2.1.2 Integridade de entidade . . . 3
2.1.3 Integridade referencial . . . 3
2.2 Chaves primárias, estrangeiras e relacionamentos . . . 3

3 Banco de dados de exemplo: Loja **4**
3.1 Inserção de dados e teste de violação . . . 5

4 Ações de integridade referencial **5**
4.1 ON DELETE CASCADE . . . 5
4.2 ON UPDATE CASCADE . . . 5
4.3 ON DELETE RESTRICT e ON UPDATE RESTRICT . . . 6
4.4 NO ACTION . . . 6
4.5 ON DELETE SET NULL . . . 6
4.6 ON UPDATE SET NULL . . . 7
4.7 Ações omitidas e regras de projeto . . . 7

5 Boas práticas e armadilhas comuns **7**

6 Exercícios **8**

7 Soluções comentadas **9**

8 Resumo final **11**

1 Conceitos iniciais: tabela pai e tabela filha

Antes de estudar as restrições de integridade referencial, é necessário compreender os papéis que duas tabelas desempenham em um relacionamento.

1.1 Tabela pai (tabela referenciada)

A **tabela pai** é aquela que contém o registro ao qual outra tabela faz referência. A coluna referenciada costuma ser sua **chave primária** (`PRIMARY KEY`), que identifica cada registro de maneira única. No exemplo de uma loja, a tabela `cliente` é pai da tabela `pedido`: cada cliente é identificado por `id_cliente`.

1.2 Tabela filha (tabela que referencia)

A **tabela filha** é aquela que contém a **chave estrangeira** (`FOREIGN KEY`) apontando para a chave da tabela pai. No exemplo, `pedido` é filha de `cliente`, porque sua coluna `id_cliente` indica a qual cliente o pedido pertence.

1.3 Exemplo introdutório

Considere os seguintes registros:

CLIENTE (pai)	
`id_cliente` (PK)	`nome`
1	Ana
2	Bruno

PEDIDO (filha)		
`id_pedido` (PK)	`id_cliente` (FK)	`data`
100	1	20/10/2026
101	1	21/10/2026
102	2	21/10/2026

Os pedidos 100 e 101 pertencem à cliente Ana, enquanto o pedido 102 pertence a Bruno. Não é permitido inserir um pedido com `id_cliente = 999`, pois esse cliente não existe na tabela pai. Se a chave estrangeira admitir `NULL`, um pedido poderá ficar sem cliente associado, conforme a regra de negócio.

```
CREATE TABLE cliente (
  id_cliente INT PRIMARY KEY,
  nome VARCHAR(100) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE pedido (
  id_pedido INT PRIMARY KEY,
  id_cliente INT NOT NULL,
  CONSTRAINT fk_pedido_cliente
    FOREIGN KEY (id_cliente) REFERENCES cliente(id_cliente)
) ENGINE=InnoDB;
```

Atenção: os termos *pai* e *filha* descrevem o papel de cada tabela em *um relacionamento específico*. Uma mesma tabela pode ser filha em um relacionamento e pai em outro. Por exemplo, `pedido` é filha de `cliente`, mas é pai de `item_pedido`.

2 Definições

A **integridade referencial** no MySQL é uma restrição que garante que os relacionamentos entre tabelas permaneçam consistentes, impedindo valores inválidos de chaves estrangeiras. Ela assegura que um registro em uma tabela filha não possa fazer referência a um registro inexistente na tabela pai.

Para implementar essa restrição, utiliza-se um mecanismo de armazenamento que ofereça suporte a chaves estrangeiras, como o **InnoDB**. Mecanismos como o MyISAM não aplicam essas restrições. O relacionamento é definido pela cláusula `FOREIGN KEY`, que associa uma ou mais colunas da tabela filha a colunas adequadamente indexadas da tabela pai, normalmente sua chave primária.

Ao definir uma chave estrangeira, podem-se especificar ações para quando o registro pai for excluído ou sua chave referenciada for alterada: `ON DELETE CASCADE` exclui os registros filhos correspondentes; `ON UPDATE CASCADE` atualiza os valores das chaves estrangeiras; `ON DELETE SET NULL` atribui `NULL` à chave estrangeira; e `RESTRICT` ou `NO ACTION` impedem a operação no registro pai enquanto houver registros filhos dependentes.

Observação: uma chave estrangeira pode aceitar `NULL`, caso a coluna seja anulável. Portanto, a regra é: cada valor *não nulo* da chave estrangeira deve corresponder a uma chave válida na tabela referenciada. Chaves estrangeiras compostas têm particularidades adicionais.

2.1 Três categorias de integridade

2.1.1 Integridade de domínio

Restringe os valores permitidos em um atributo. Por exemplo, uma quantidade de itens deve ser positiva: `quantidade INT NOT NULL CHECK (quantidade > 0)`.

2.1.2 Integridade de entidade

Cada linha deve ser identificável por sua chave primária. Uma `PRIMARY KEY` é única e não aceita `NULL`.

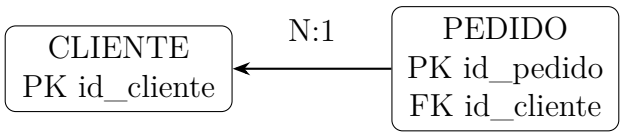
2.1.3 Integridade referencial

As chaves estrangeiras preservam a coerência entre tabelas relacionadas. Por exemplo, um pedido não pode apontar para um cliente inexistente.

2.2 Chaves primárias, estrangeiras e relacionamentos

Chave primária (PK): atributo ou conjunto de atributos que identifica exclusivamente cada registro. **Chave estrangeira (FK):** atributo ou conjunto de atributos da tabela filha que referencia uma chave da tabela pai.

No relacionamento **1:N**, um cliente pode ter vários pedidos, mas cada pedido pertence a um cliente. No relacionamento **1:1**, uma FK com `UNIQUE` pode limitar cada pai a um único filho. No relacionamento **N:N**, usa-se uma tabela associativa, como `item_pedido`, com duas chaves estrangeiras.



3 Banco de dados de exemplo: Loja

O banco possui clientes, pedidos, produtos e itens de pedidos. A tabela `item_pedido` associa pedidos a produtos, representando um relacionamento N:N entre eles.

```
CREATE DATABASE IF NOT EXISTS loja_integridade;
USE loja_integridade;

CREATE TABLE cliente (
  id_cliente INT PRIMARY KEY,
  nome VARCHAR(100) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE produto (
  id_produto INT PRIMARY KEY,
  descricao VARCHAR(100) NOT NULL,
  preco DECIMAL(10,2) NOT NULL,
  CONSTRAINT ck_preco CHECK (preco >= 0)
) ENGINE=InnoDB;

CREATE TABLE pedido (
  id_pedido INT PRIMARY KEY,
  id_cliente INT NOT NULL,
  data_pedido DATE NOT NULL,
  CONSTRAINT fk_pedido_cliente
    FOREIGN KEY (id_cliente) REFERENCES cliente(id_cliente)
    ON DELETE RESTRICT
    ON UPDATE CASCADE
) ENGINE=InnoDB;

CREATE TABLE item_pedido (
  id_pedido INT NOT NULL,
  numero_item INT NOT NULL,
  id_produto INT NOT NULL,
  quantidade INT NOT NULL,
  preco_unitario DECIMAL(10,2) NOT NULL,
  PRIMARY KEY (id_pedido, numero_item),
  CONSTRAINT ck_quantidade CHECK (quantidade > 0),
  CONSTRAINT ck_preco_item CHECK (preco_unitario >= 0),
  CONSTRAINT fk_item_pedido FOREIGN KEY (id_pedido)
    REFERENCES pedido(id_pedido)
    ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT fk_item_produto FOREIGN KEY (id_produto)
    REFERENCES produto(id_produto)
    ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;
```

Interpretação das regras: não se pode excluir um cliente que tenha pedidos; excluir um pedido exclui seus itens; não se pode excluir um produto presente em itens de pedidos; alterar o identificador de cliente, pedido ou produto propaga a alteração às FKs correspondentes. Em aplicações reais, alterar identificadores primários costuma ser raro.

3.1 Inserção de dados e teste de violação

```
INSERT INTO cliente VALUES (1, 'Ana'), (2, 'Bruno');
INSERT INTO produto VALUES (10, 'Livro', 45.00),
                           (20, 'Jogo', 90.00);
INSERT INTO pedido VALUES (100, 1, '2026-10-20');
INSERT INTO item_pedido VALUES (100, 1, 10, 2, 45.00);

-- Erro: cliente 999 nao existe.
INSERT INTO pedido VALUES (101, 999, '2026-10-21');
```

O último comando viola a integridade referencial. O SGBD rejeita a inserção, pois não há linha pai com `id_cliente=999`.

4 Ações de integridade referencial

As ações referenciais são especificadas separadamente para `DELETE` e `UPDATE` na tabela pai. Não são ações disparadas por qualquer atualização: `ON UPDATE` diz respeito à alteração das *colunas referenciadas*.

Ação	Efeito quando há filhos referenciando o pai
`CASCADE`	Propaga a exclusão ou atualização aos filhos.
`RESTRICT`	Rejeita a operação que quebraria a referência.
`NO ACTION`	No InnoDB, comporta-se como `RESTRICT`; a verificação não é adiada até o `COMMIT`.
`SET NULL`	Define a FK dos filhos como `NULL`; exige colunas anuláveis.
`SET DEFAULT`	Não é uma ação suportada para uso efetivo em FKs InnoDB no MySQL.

4.1 ON DELETE CASCADE

Ao excluir o pedido 100, seus itens são excluídos automaticamente, pois a FK de `item_pedido` aponta para `pedido` com essa ação.

```
-- Execute depois dos testes de consulta desejados:
DELETE FROM pedido WHERE id_pedido = 100;
SELECT * FROM item_pedido WHERE id_pedido = 100;
-- Resultado: nenhuma linha.
```

A exclusão não remove o cliente nem o produto. O sentido da propagação é *pai para filho*.

4.2 ON UPDATE CASCADE

Recrie o pedido e o item caso tenha executado a exclusão anterior.

```
INSERT INTO pedido VALUES (100, 1, '2026-10-20');
INSERT INTO item_pedido VALUES (100, 1, 10, 2, 45.00);
UPDATE cliente SET id_cliente = 5 WHERE id_cliente = 1;
SELECT id_pedido, id_cliente FROM pedido;
-- O pedido 100 passa a apontar para o cliente 5.
```

O registro do pedido não é excluído: somente a FK é atualizada. Uma atualização do nome do cliente não aciona a propagação da chave.

4.3 ON DELETE RESTRICT e ON UPDATE RESTRICT

```
-- Falha: o cliente 5 ainda possui o pedido 100.
DELETE FROM cliente WHERE id_cliente = 5;
```

Se uma FK usar `ON UPDATE RESTRICT`, a alteração de uma chave pai referenciada também será recusada. `RESTRICT` protege registros dependentes de referências quebradas.

4.4 NO ACTION

```
CREATE TABLE categoria (
  id_categoria INT PRIMARY KEY,
  nome VARCHAR(60) NOT NULL
) ENGINE=InnoDB;
CREATE TABLE artigo (
  id_artigo INT PRIMARY KEY,
  id_categoria INT NOT NULL,
  FOREIGN KEY (id_categoria) REFERENCES categoria(id_categoria)
    ON DELETE NO ACTION ON UPDATE NO ACTION
) ENGINE=InnoDB;
```

No MySQL/InnoDB, `NO ACTION` e `RESTRICT` têm o mesmo efeito prático: não há verificação diferida de FK. Em outros SGBDs, `NO ACTION` pode ter semântica diferente quando há restrições adiáveis.

4.5 ON DELETE SET NULL

Uma tarefa pode existir sem estar atribuída a um funcionário. Nesse caso, a FK deve permitir `NULL`.

```
CREATE TABLE funcionario (
  id_funcionario INT PRIMARY KEY,
  nome VARCHAR(100) NOT NULL
) ENGINE=InnoDB;
CREATE TABLE tarefa (
  id_tarefa INT PRIMARY KEY,
  id_funcionario INT NULL,
  descricao VARCHAR(100) NOT NULL,
  FOREIGN KEY (id_funcionario)
    REFERENCES funcionario(id_funcionario)
    ON DELETE SET NULL ON UPDATE CASCADE
```

```
) ENGINE=InnoDB;
INSERT INTO funcionario VALUES (7, 'Carla');
INSERT INTO tarefa VALUES (50, 7, 'Revisar estoque');
DELETE FROM funcionario WHERE id_funcionario = 7;
SELECT * FROM tarefa;
-- A tarefa 50 continua existindo; id_funcionario = NULL.
```

Se `id_funcionario` fosse `NOT NULL`, a definição da FK com `SET NULL` seria inválida.

4.6 ON UPDATE SET NULL

Também é possível configurar `ON UPDATE SET NULL`, desde que a FK seja anulável. Se a chave referenciada mudar, a referência no filho passa a `NULL`, em vez de acompanhar a nova chave.

```
CREATE TABLE equipamento (
  id_equipamento INT PRIMARY KEY,
  nome VARCHAR(60) NOT NULL
) ENGINE=InnoDB;
CREATE TABLE reserva (
  id_reserva INT PRIMARY KEY,
  id_equipamento INT NULL,
  FOREIGN KEY (id_equipamento)
    REFERENCES equipamento(id_equipamento)
    ON DELETE SET NULL ON UPDATE SET NULL
) ENGINE=InnoDB;
INSERT INTO equipamento VALUES (30, 'Projetor');
INSERT INTO reserva VALUES (200, 30);
UPDATE equipamento SET id_equipamento = 31
WHERE id_equipamento = 30;
-- A reserva 200 passa a ter id_equipamento = NULL.
```

4.7 Ações omitidas e regras de projeto

Se `ON DELETE` ou `ON UPDATE` for omitido, o comportamento padrão do InnoDB é equivalente a `NO ACTION`/`RESTRICT`. Escolha a ação com base na **regra de negócio**, não apenas na conveniência de executar comandos.

5 Boas práticas e armadilhas comuns

- Use InnoDB nas tabelas envolvidas. MyISAM não aplica FKs.
- Os tipos das colunas relacionadas devem ser compatíveis; para inteiros, observe também `SIGNED`/`UNSIGNED`.
- A coluna referenciada deve ter índice apropriado. Prefira referenciar `PRIMARY KEY` ou `UNIQUE`; referências a chaves não únicas são desaconselhadas e sujeitas a restrições de versão/configuração.
- A FK da tabela filha precisa de índice; o InnoDB pode criá-lo automaticamente se necessário.

- `CASCADE` pode afetar muitas linhas. Faça testes antes de utilizá-lo em produção.
- `SET NULL` representa perda permitida do vínculo, não exclusão do filho.
- A ausência de uma FK não impede criar colunas com nomes parecidos, mas deixa a consistência sem garantia automática.
- `FOREIGN KEY` não substitui regras de negócio mais complexas, que podem exigir transações, verificações ou lógica adicional.

6 Exercícios

Orientação: em cada exercício, escreva o `CREATE TABLE` com PKs, FKs, tipos, nulidade e ações referenciais. Justifique as escolhas e indique o resultado das operações propostas.

Exercício 1 – Biblioteca

Uma editora publica muitos livros, e cada livro pertence a uma única editora. A biblioteca não permite excluir editoras que possuam livros cadastrados. Se o código de uma editora mudar, todos os livros devem acompanhar a mudança. Crie `editora` e `livro` e especifique as ações. O que ocorre ao tentar excluir uma editora com livros?

Exercício 2 – Escola

Uma turma possui vários alunos. Quando uma turma é excluída, seus alunos devem permanecer no sistema, temporariamente sem turma. O código da turma, se alterado, deve ser propagado. Crie `turma` e `aluno`. Explique por que a FK deve aceitar `NULL`.

Exercício 3 – Comércio eletrônico

Um pedido contém vários itens; cada item pertence a exatamente um pedido. Ao cancelar definitivamente um pedido por exclusão física, seus itens devem ser removidos. Crie `pedido` e `item`, identificando cada item pelo par (pedido, número sequencial). Mostre o efeito de excluir um pedido.

Exercício 4 – Departamento e empregado

Cada empregado pertence a um departamento. O departamento não pode ser excluído enquanto tiver empregados. Também não é permitido alterar seu identificador se estiver referenciado. Crie as tabelas usando explicitamente `RESTRICT` em ambas as ações. Demonstre uma operação rejeitada.

Exercício 5 – Curso e matrícula

Um estudante pode se matricular em vários cursos, e um curso pode ter vários estudantes. Cada matrícula possui data e nota. Se um estudante for excluído, suas matrículas devem ser excluídas; não se pode excluir um curso que possua matrículas. Alterações dos identificadores devem ser propagadas. Crie as três tabelas e explique a tabela associativa.

7 Soluções comentadas

Solução 1 – Biblioteca

```
CREATE TABLE editora (
 id_editora INT PRIMARY KEY,
 nome VARCHAR(100) NOT NULL
) ENGINE=InnoDB;
CREATE TABLE livro (
 id_livro INT PRIMARY KEY,
 titulo VARCHAR(150) NOT NULL,
 id_editora INT NOT NULL,
 FOREIGN KEY (id_editora) REFERENCES editora(id_editora)
 ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;
```

Comentário: `RESTRICT` impede excluir uma editora referenciada; `CASCADE` atualiza o identificador nos livros. Não é permitido inserir livro com editora inexistente.

Solução 2 – Escola

```
CREATE TABLE turma (
 id_turma INT PRIMARY KEY,
 nome VARCHAR(60) NOT NULL
) ENGINE=InnoDB;
CREATE TABLE aluno (
 id_aluno INT PRIMARY KEY,
 nome VARCHAR(100) NOT NULL,
 id_turma INT NULL,
 FOREIGN KEY (id_turma) REFERENCES turma(id_turma)
 ON DELETE SET NULL ON UPDATE CASCADE
) ENGINE=InnoDB;
```

Comentário: ao excluir a turma, seus alunos continuam cadastrados com `id_turma=NULL`. A coluna não pode ser `NOT NULL`, pois o próprio comportamento desejado requer a ausência temporária de turma.

Solução 3 – Comércio eletrônico

```
CREATE TABLE pedido_ex3 (
 id_pedido INT PRIMARY KEY,
 data_pedido DATE NOT NULL
) ENGINE=InnoDB;
CREATE TABLE item_ex3 (
 id_pedido INT NOT NULL,
 numero_item INT NOT NULL,
 descricao VARCHAR(100) NOT NULL,
 quantidade INT NOT NULL CHECK (quantidade > 0),
 PRIMARY KEY (id_pedido, numero_item),
 FOREIGN KEY (id_pedido) REFERENCES pedido_ex3(id_pedido)
```

```
 ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;
INSERT INTO pedido_ex3 VALUES (1, '2026-10-20');
INSERT INTO item_ex3 VALUES (1, 1, 'Caderno', 2);
DELETE FROM pedido_ex3 WHERE id_pedido = 1;
-- O item (1,1) tambem e removido.
```

Comentário: a PK composta permite repetir o número do item em pedidos diferentes, mas não dentro do mesmo pedido. A FK faz parte da PK e, portanto, não pode ser nula.

Solução 4 – Departamento e empregado

```
CREATE TABLE departamento (
 id_departamento INT PRIMARY KEY,
 nome VARCHAR(80) NOT NULL
) ENGINE=InnoDB;
CREATE TABLE empregado (
 id_empregado INT PRIMARY KEY,
 nome VARCHAR(100) NOT NULL,
 id_departamento INT NOT NULL,
 FOREIGN KEY (id_departamento)
 REFERENCES departamento(id_departamento)
 ON DELETE RESTRICT ON UPDATE RESTRICT
) ENGINE=InnoDB;
INSERT INTO departamento VALUES (10, 'Financeiro');
INSERT INTO empregado VALUES (1, 'Rita', 10);
DELETE FROM departamento WHERE id_departamento = 10;
-- Rejeitado: existe empregado referenciando 10.
UPDATE departamento SET id_departamento = 11
 WHERE id_departamento = 10;
-- Tambem rejeitado.
```

Comentário: para excluir o departamento, primeiro seria necessário realocar ou excluir os empregados. As duas ações preservam a referência recusando alterações no pai.

Solução 5 – Curso e matrícula

```
CREATE TABLE estudante (
 id_estudante INT PRIMARY KEY,
 nome VARCHAR(100) NOT NULL
) ENGINE=InnoDB;
CREATE TABLE curso (
 id_curso INT PRIMARY KEY,
 titulo VARCHAR(100) NOT NULL
) ENGINE=InnoDB;
CREATE TABLE matricula (
 id_estudante INT NOT NULL,
 id_curso INT NOT NULL,
 data_matricula DATE NOT NULL,
 nota DECIMAL(4,2),
 PRIMARY KEY (id_estudante, id_curso),
```

```
 FOREIGN KEY (id_estudante) REFERENCES estudante(id_estudante)
 ON DELETE CASCADE ON UPDATE CASCADE,
 FOREIGN KEY (id_curso) REFERENCES curso(id_curso)
 ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;
```

Comentário: `matricula` representa o relacionamento N:N e armazena atributos próprios da associação. A PK composta admite uma matrícula por par estudante–curso; se forem permitidas matrículas repetidas em semestres diferentes, o período deve integrar a identificação ou deve ser criado um identificador próprio. A exclusão de estudante apaga suas matrículas, mas não os cursos; a exclusão de curso com matrículas é impedida.

8 Resumo final

A `PRIMARY KEY` identifica linhas; a `FOREIGN KEY` estabelece e verifica referências; e as ações `ON DELETE/ON UPDATE` determinam como lidar com a exclusão ou alteração de uma chave pai referenciada. `CASCADE` propaga mudanças, `SET NULL` desfaz a associação sem apagar o filho, e `RESTRICT/NO ACTION` bloqueiam mudanças que violariam a referência. A decisão correta depende sempre das regras de negócio.